CS917 Algorithms Coursework Answer

YuChin Chiang
ID: u5693885

Question 1

The question gives time limits, and we need to find out the largest input size n, provided that an algorithm takes $f(n)$ microseconds to finish. Formally, we seek the largest integer n satisfying:

$$f(n) \leq t,$$

where t is the total available time (μs).

Time conversions

1 second	1×10^6 μs
1 minute	6×10^7 μs
1 hour	3.6×10^9 μs
1 day	8.64×10^{10} μs
1 month (30 days)	2.592×10^{12} μs
1 year (365 days)	3.1536×10^{13} μs

Listing 1: Binary search for largest n

```
import math

times = {"1s":1e6,"1m":60e6,"1h":3600e6,
         "1d":86400e6,"1mo":30*86400e6,
         "1y":365*86400e6}

funcs = {"n log2 n": lambda n: n*math.log2(n),
         "n^2": lambda n: n**2,
         "n^3": lambda n: n**3,
         "2^n": lambda n: 2**n,
         "n!": lambda n: math.factorial(n)}

def max_n(f, t, low=1, high=None):
    n = 1
    while high is None:
        try:
            if f(n) > t:
                high = n
                break
            n *= 2
        except (OverflowError, ValueError):
            high = n
            break
    low = n // 2
    while low < high - 1:
        mid = (low + high) // 2
        try:
            if f(mid) <= t:
                low = mid
            else:
                high = mid
        except (OverflowError, ValueError):
            high = mid
    return low
```

Estimated results

$f(n)$	1 s	1 m	1 h	1 d	1 mo	1 y
$n \log_2 n$	6.3×10^4	2.8×10^6	1.3×10^8	2.8×10^9	8.2×10^{10}	9.7×10^{11}
n^2	1.0×10^3	7.7×10^3	6.0×10^4	2.9×10^5	1.6×10^6	5.6×10^6
n^3	100	391	1.5×10^3	4.4×10^3	1.4×10^4	3.1×10^4
2^n	20	26	32	37	41	44
$n!$	9	11	12	13	15	16

Question 2

Definitions

- $f(n) \in O(g(n))$ if $\exists\, c > 0, n_0 > 0$ such that $0 \le f(n) \le c\, g(n)$ for all $n \ge n_0$.
- $f(n) \in \Omega(g(n))$ if $\exists\, c > 0, n_0 > 0$ such that $0 \le c\, g(n) \le f(n)$ for all $n \ge n_0$.
- $f(n) \in \Theta(g(n))$ if $f(n)$ is both $O(g(n))$ and $\Omega(g(n))$.

(a) $9n^2 + 9 \in O(n^2)$

For $n \ge 1$,

$$9n^2 + 9 \le 9n^2 + n^2 = 10n^2.$$

Thus we may choose $c = 10$ and $n_0 = 1$, satisfying the Big-O definition. $\boxed{\text{True}}$

(b) $6n^4 - 3n^2 + 3 \in \Omega(n^4)$

For $n \ge 1$,

$$6n^4 - 3n^2 + 3 \ge 6n^4 - 3n^4 = 3n^4.$$

Thus $c = 3$ and $n_0 = 1$ satisfy the Big-Ω definition. $\boxed{\text{True}}$

(c) $9n^3 - 7n \in O(n^4)$

Since $n^3 \le n^4$ for $n \ge 1$, we have

$$9n^3 - 7n \le 9n^3 \le 9n^4.$$

Thus $c = 9$ and $n_0 = 1$ satisfy the Big-O definition. $\boxed{\text{True}}$

(d) $n^4 + 2n^2 + 1 \in \Theta(n^2)$

To be $\Theta(n^2)$, the function must be both $O(n^2)$ and $\Omega(n^2)$. Although

$$n^4 + 2n^2 + 1 \ge n^2,$$

so the $\Omega(n^2)$ condition holds, it fails the $O(n^2)$ condition:

$$\frac{n^4 + 2n^2 + 1}{n^2} = n^2 + 2 + \frac{1}{n^2} \to \infty.$$

Thus no constant c can satisfy $f(n) \le cn^2$ for large n. $\boxed{\text{False}}$

(e) $n^2 + 6n + 143 \in \Omega(n)$

Since $n^2 \geq n$ for all $n \geq 1$, we have

$$n^2 + 6n + 143 \geq n^2 \geq n.$$

Thus $c = 1$, $n_0 = 1$ satisfy the Big-Ω definition. $\boxed{\text{True}}$

Item	Expression	Result	Reason
(a)	$9n^2 + 9 \in O(n^2)$	True	$9n^2 + 9 \leq 10n^2$.
(b)	$6n^4 - 3n^2 + 3 \in \Omega(n^4)$	True	$6n^4 - 3n^2 + 3 \geq 3n^4$.
(c)	$9n^3 - 7n \in O(n^4)$	True	$9n^3 - 7n \leq 9n^4$.
(d)	$n^4 + 2n^2 + 1 \in \Theta(n^2)$	False	Not $O(n^2)$; ratio $\to \infty$.
(e)	$n^2 + 6n + 143 \in \Omega(n)$	True	$n^2 + 6n + 143 \geq n$.

Question 3

Where possible, apply the Master Theorem for the following recurrences. In each case, state whether the Master Theorem applies, and if so, give a tight $\Theta(\cdot)$ bound for $T(n)$. If it does not apply, briefly explain why.

Solution

We compare each recurrence to the form

$$T(n) = a\,T(n/b) + f(n),$$

where the Master Theorem assumes $a \geq 1$, $b > 1$ are *constants* and the subproblem size n/b is smaller than n.

(a) $T(n) = T(4n/3) + 5$

Here the subproblem has size $\frac{4n}{3} > n$, so the problem size *increases* at each step. This violates the requirement $b > 1$ with $n/b < n$.

$$\boxed{\text{Master Theorem does not apply (subproblem gets larger).}}$$

(b) $T(n) = 7T(n/2) + 47n$

We have $a = 7$, $b = 2$, so

$$n^{\log_b a} = n^{\log_2 7}.$$

The term $f(n) = 47n = \Theta(n)$ satisfies

$$f(n) = O\big(n^{\log_2 7 - \varepsilon}\big)$$

for some $\varepsilon > 0$, since $\log_2 7 \approx 2.807 > 1$. Thus this is **Case 1** of the Master Theorem.

$$\boxed{T(n) = \Theta\big(n^{\log_2 7}\big).}$$

(c) $T(n) = 2^n T(n/2) + n$

In the Master Theorem, a must be a constant. Here $a = 2^n$ depends on n, so the theorem's conditions are not satisfied.

$$\boxed{\text{Master Theorem does not apply } (a \text{ is not constant}).}$$

(d) $T(n) = 16T(n/4) + 16n^2$

Here $a = 16$, $b = 4$, so

$$n^{\log_b a} = n^{\log_4 16} = n^2.$$

We have $f(n) = 16n^2 = \Theta(n^2) = \Theta\big(n^{\log_4 16}\big)$, so this matches **Case 2** of the Master Theorem with $k = 0$.

$$\boxed{T(n) = \Theta(n^2 \log n).}$$

(e) $T(n) = 4T(5n/3) + 50n^3$

The subproblem size is $\frac{5n}{3} > n$, so again the problem size increases at each step. In the form $T(n) = aT(n/b) + f(n)$ we would have $b = \frac{3}{5} < 1$, which is not allowed by the Master Theorem.

$$\boxed{\text{Master Theorem does not apply (subproblem gets larger).}}$$

Item	**Recurrence**	**MT?**	**Result**
(a)	$T(n) = T(4n/3) + 5$	No	subproblem larger
(b)	$T(n) = 7T(n/2) + 47n$	Yes	$\Theta\big(n^{\log_2 7}\big)$
(c)	$T(n) = 2^n T(n/2) + n$	No	a not constant
(d)	$T(n) = 16T(n/4) + 16n^2$	Yes	$\Theta(n^2 \log n)$
(e)	$T(n) = 4T(5n/3) + 50n^3$	No	subproblem larger

Question 4

For each list, we choose the most appropriate sorting algorithm based on its structure, expected running time, memory usage, and whether stability is required.

(a) [1, 2, 3, 4, 6, 5, 7, 10]

This list is almost completely sorted, with only a small local inversion. Insertion Sort is the most effective algorithm in this case: it runs in $\Theta(n)$ time on nearly sorted inputs, requires only $O(1)$ extra space, and performs only a few element shifts.

$$\boxed{\text{Insertion Sort}}$$

(b) [13, 12, 9, 6, 5, 4, 3, 2, 1]

The list is in strictly decreasing order, which is the worst-case input for Insertion Sort and naive Quick Sort. A divide-and-conquer algorithm with guaranteed worst-case behaviour, such as Merge Sort, is more suitable. It always runs in $\Theta(n \log n)$ time regardless of input order.

$$\boxed{\text{Merge Sort}}$$

(c) [1, 10, 5, 8, 3, 9, 6, 4, 2]

This list appears to be an unsorted, random-order input. For general-purpose sorting, Quick Sort with a good pivot strategy (e.g. randomized or median-of-three) is typically the most efficient, achieving an average-case time of $\Theta(n \log n)$ with small constants and in-place behaviour.

$$\boxed{\text{Quick Sort}}$$

(d) [6, 5, 6, 5, 9, 11, 1, 23, 7]
Stability required: equal-valued elements must preserve order.

Because stability is explicitly required, algorithms such as Heap Sort and standard Quick Sort are unsuitable, as they are not stable in their typical implementations. A stable $\Theta(n \log n)$ algorithm, such as Merge Sort, is appropriate and preserves the relative order of elements with the same value.

$$\boxed{\text{Stable Merge Sort}}$$

(e) [(3,9), (4,5), (4,4), (9,5), (8,7), (10,6)]
Sort by key, preserving value order for identical keys.

Each item contains a key and a value, and when keys are equal, the values must remain in their original order. This again requires a stable sorting algorithm. Stable Merge Sort will sort by the key while preserving the relative positions of equal-key pairs.

$$\boxed{\text{Stable Merge Sort (sorting by key)}}$$

Question 5

Apply Dijkstra's Algorithm to the given graph, computing the shortest paths from vertex A. At each stage we select the unvisited vertex with the smallest tentative distance and relax all outgoing edges. The table records the vertex added to the search tree and the updated (predecessor, distance) labels for each vertex.

Stage	Current Vertex	Labels and Distances					
		A	B	C	D	E	F
0	–	A 0	A 17	A 47	∞	∞	∞
1	A	A 0	A 17	A 47	∞	∞	∞
2	B	A 0	A 17	A 47	B 28	B 24	∞
3	E	A 0	A 17	E 39	B 28	B 24	E 33
4	D	A 0	A 17	E 39	B 28	B 24	E 33
5	F	A 0	A 17	E 39	B 28	B 24	E 33
6	C	A 0	A 17	E 39	B 28	B 24	E 33

TABLE I: Stages of Dijkstra's Algorithm from vertex A

Explanation

Starting with a vertex A labelled $(A, 0)$. From A, we relax edges to obtain tentative distances $d(B) = 17$ and $d(C) = 47$. Vertex B is then selected as the smallest unvisited label, and relaxing its edges updates $d(E) = 24$ and $d(D) = 28$. Next, a vertex E is chosen, and relaxation yields $d(F) = 33$ and improves $d(C)$ to 39 via $A \to B \to E \to C$. Vertices D, F, and finally C are then settled, with no further improvements. Vertices are processed in increasing order of final distance.

The final distances from A are:

$$\boxed{d(A) = 0 \quad d(B) = 17 \quad d(E) = 24 \quad d(D) = 28 \quad d(F) = 33 \quad d(C) = 39}$$

The corresponding shortest paths are:

$$\boxed{\begin{array}{ll} A \to B & (17) \\ A \to B \to E & (24) \\ A \to B \to D & (28) \\ A \to B \to E \to F & (33) \\ A \to B \to E \to C & (39) \end{array}}$$

Question 6

We apply Dijkstra's Algorithm on the directed graph starting from vertex A. The intermediate stages are shown in the table.

Stage	Current	A	B	C	D	E	F
0	A	(A,0)	(∞)	(∞)	(∞)	(∞)	(∞)
1	A	(A,0)	(A,13)	(∞)	(∞)	(∞)	(∞)
2	B	(A,0)	(A,13)	(∞)	(∞)	(B,28)	(∞)
3	E	(A,0)	(A,13)	(∞)	(∞)	(B,28)	(E,42)
4	F	(A,0)	(A,13)	(∞)	(∞)	(B,28)	(E,42)

Explanation

Since the graph is directed, only outgoing edges can be used during the execution. As the graph is directed, your only choice would be to use outgoing edges while Dijkstra's Algorithm is being run. So let's start at node A with distance 0. B is the only vertex reachable from A along 13 and will be added to st. From B, the distance of E is updated as 28 over $A \to B \to E$. Next, from E, we visit vertex F with distance 42 via the path $A \to B \to E \to F$. Relaxing vertex F does not help E since the one that is there (28) is already less than $42 + 6$. We also see that vertices C and D do not have a directed path from A, so they are kept at infinite distance. The net distances from A are:

$$\boxed{d(A) = 0 \quad d(B) = 13 \quad d(E) = 28 \quad d(F) = 42 \quad d(C) = \infty \quad d(D) = \infty}$$

Question 7

(a) Prim's Algorithm

Applying Prim's Algorithm from vertex A, each stage adds one new vertex and one new edge to the MST $S = \{V, E\}$. At every step, we select the smallest edge leaving the current vertex set V.

Stage	V	E
0	(A)	()
1	(A,B)	((A,B))
2	(A,B,C)	((A,B),(B,C))
3	(A,B,C,G)	((A,B),(B,C),(C,G))
4	(A,B,C,G,F)	((A,B),(B,C),(C,G),(G,F))
5	(A,B,C,G,F,E)	((A,B),(B,C),(C,G),(G,F),(F,E))
6	(A,B,C,G,F,E,D)	((A,B),(B,C),(C,G),(G,F),(F,E),(E,D))

TABLE II: Stages of Prim's Algorithm starting from A

Explanation

Begin with $V = \{A\}$. At each stage, we select the cheapest edge connecting V to a new vertex. The choices, in order, are:

$$A \to B\ (5) \quad B \to C\ (2) \quad C \to G\ (5) \quad G \to F\ (9) \quad F \to E\ (1) \quad E \to D\ (8)$$

These edges grow the MST one vertex at a time.

$$\boxed{\{(A,B),(B,C),(C,G),(G,F),(F,E),(E,D)\}}$$

(b) Kruskal's Algorithm

Kruskal's Algorithm processes edges in non-decreasing order of weight. At each stage, one edge is inspected. If it connects two different components, it is added to E.

Stage	Edge	Components	E
0	–	(A)(B)(C)(D)(E)(F)(G)	()
1	(E,F)	(A)(B)(C)(D)(E,F)(G)	((E,F))
2	(B,C)	(A)(B,C)(D)(E,F)(G)	((E,F),(B,C))
3	(A,B)	(A,B,C)(D)(E,F)(G)	((E,F),(B,C),(A,B))
4	(C,G)	(A,B,C,G)(D)(E,F)	((E,F),(B,C),(A,B),(C,G))
5	(D,E)	(A,B,C,G)(D,E,F)	((E,F),(B,C),(A,B),(C,G),(D,E))
6	(F,G)	(A,B,C,G,D,E,F)	((E,F),(B,C),(A,B),(C,G),(D,E),(F,G))

TABLE III: Stages of Kruskal's Algorithm

Explanation

Edges are examined in increasing order of weight. An edge is added only if it joins two different components. The selected edges, in order, are:

$$E \to F\ (1)\ B \to C\ (2)\ A \to B\ (5)\ C \to G\ (5)\ D \to E\ (8)\ F \to G\ (9)$$

Once all vertices lie in a single component, the MST is complete.

$$\boxed{\{(E,F),(B,C),(A,B),(C,G),(D,E),(F,G)\}}$$

Question 8

Figure 1 illustrates the effect of performing a left rotation and a right rotation in a red-black tree. A left rotation at node x transforms the configuration on the left into the configuration on the right, and a right rotation is its inverse. The subtrees α, β, and γ are arbitrary subtrees.

The pseudo-code for LEFT-ROTATE is given in Algorithm 1. Using the same conventions for parent and child pointers, we complete the symmetric operation RIGHT-ROTATE in Algorithm 2.

Explanation

A left rotation moves the right child y of x upward to replace x, while x becomes the left child of y. The subtree $\beta = y$.left is transferred to become the new right child of x.

The RIGHT-ROTATE operation is the exact mirror image. The left child x of y is promoted to take y's place, and y becomes the right child of x. The subtree $\beta = x$.right is transferred to become the left subtree of y. This preserves the binary-search-tree ordering property and requires only a constant number of pointer changes.

Algorithm 1 LEFT-ROTATE(T, x)

```
y ← x.right
x.right ← y.left
if y.left ≠ T.nil then
    y.left.p ← x
end if
y.p ← x.p
if x.p == T.nil then
    T.root ← y
else if x == x.p.left then
    x.p.left ← y
else
    x.p.right ← y
end if
y.left ← x
x.p ← y
```

Algorithm 2 RIGHT-ROTATE(T, y)

```
x ← y.left
y.left ← x.right
if x.right ≠ T.nil then
    x.right.p ← y
end if
x.p ← y.p
if y.p == T.nil then
    T.root ← x
else if y == y.p.right then
    y.p.right ← x
else
    y.p.left ← x
end if
x.right ← y
y.p ← x
```

Question 9

This question concerns decoding Morse code and navigating a maze. Below describe the design choices, algorithms, and complexity analysis for each required function.

(a) Morse Code Decode

The input is a list of strings, where each string is a Morse-encoded letter (consisting only of dots (`.`) and dashes (`-`)). To decode the sentence, we simply look up each Morse sequence in a dictionary that maps Morse patterns to their corresponding English letters. For example:

$$\texttt{'.-'} \mapsto A, \quad \texttt{'...'} \mapsto S, \quad \texttt{'--'} \mapsto M.$$

The algorithm scans the input list from left to right, performs a dictionary lookup for each Morse string, and appends the decoded letter to the output word.

a) Algorithm.:

- Construct a Python dictionary mapping every Morse sequence to its letter.
- For each Morse string in the input list, perform a dictionary lookup.
- Append the returned letter to a result list and concatenate at the end.

b) Time Complexity.:

$$\boxed{T(k) = O(k)}$$

(b) Incomplete Morse Code

In this task, the first character of every Morse string is unknown and is represented by the symbol `x`. For example, the word:

$$\text{“TEST”} \longrightarrow ['x', 'x', 'x..', 'x'].$$

Since each `x` can represent either `.` or `-`, every incomplete Morse letter may expand into up to two valid Morse patterns. Thus, for a word of length n, the total number of candidate Morse sequences may be:

$$2^n.$$

To determine which of these candidate words form valid English words, the algorithm proceeds as follows.

c) Algorithm.:

leftmirgin=2cm Load all valid English words from `dictionary.txt`.
leftmiirgiin=2cm For each incomplete Morse string, generate all possible expansions by replacing `x` with both `.` and `-`.
leftmiiirgiiin=2cm Form the Cartesian product of all expansions to obtain every possible decoded word.
leftmivrgivn=2cm Check each decoded string against the dictionary.
leftmvrgvn=2cm Return the list of all valid English words.

d) Example.: For the input:

$$['x..', 'x-', 'x.', 'x.-.', 'x']$$

the method successfully returns:

$$\boxed{\{\text{DANCE},\ \text{SANCT}\}}$$

e) Time Complexity.: Each incomplete letter may branch into two possibilities, so the worst-case number of candidate words is:

$$\boxed{T(n) = O(2^n)}$$

(c) The Maze

The maze is represented as a two-dimensional grid, where:

$$0 = \text{open space}, \qquad 1 = \text{wall}.$$

Only coordinates provided using `addCoordinate` are explicitly known. All unspecified coordinates inside the bounding rectangle are treated as walls.

addCoordinate(x, y, blockType): This method stores:

$$\text{maze}[y][x] = \text{blockType},$$

and also updates:

$$\text{maxX} = \max(\text{maxX}, x), \qquad \text{maxY} = \max(\text{maxY}, y).$$

printMaze(): The maze is printed row-by-row using:

$$\star \text{ for walls,} \qquad \text{(space) for open cells.}$$

findRoute(x_1, y_1, x_2, y_2): To find a route from the start coordinate to the target coordinate, I apply Breadth-First Search (BFS), which is the standard algorithm for shortest paths in an unweighted grid.

f) BFS Rules.:

- Only open (`0`) cells may be traversed.
- Movement is allowed up, down, left, and right.
- A queue is used to explore all reachable neighbours.
- A `parent` dictionary records the previous coordinate for reconstructing the final path.

g) Output.: The method returns a list of coordinates:

$$[(x_1, y_1), (x_2, y_2), \ldots, (x_n, y_n)]$$

representing the full route. If no path exists, an empty list is returned.

h) Example Result.:

$$\boxed{(1,0) \to (1,1) \to (2,1) \to \cdots \to (3,8)}$$

i) Time Complexity.: With a grid of size $M \times N$, BFS runs in:

$$\boxed{O(MN)}$$